

作业 12

——(1+eps)欧氏近似最近点查询——

声明：题目来源于 <http://openjudge.cn/>，解题思路参考教授上课讲解内容，代码实现由本人完成。

分享这个文档仅限于学术交流，不用于任何形式的商业用途。

1. 题目

描述

给出 n 个二维平面上的数据点 $P = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ 。

你的程序需要回答 q 次询问，每次询问给出一个二维平面上的点 (a_i, b_i) ，询问集合 P 中距离 (a_i, b_i) 最近的数据点。

假设集合 P 中距离 (a_i, b_i) 最近的数据点到 (a_i, b_i) 的距离为 $d((a_i, b_i), P)$ ，你需要输出一个 $[1, n]$ 中的整数 i ，使得 $d((a_i, b_i), (x_{J_i}, y_{J_i})) \leq 1.15 * d((a_i, b_i), P)$ 。

保证查询点到最近数据点的距离如果非零，则至少是 1。

输入

第一行一个整数 $n, q (1 \leq n, q \leq 10^5)$ 。

接下来的 n 行，每行两个浮点数 $x_i, y_i (0 \leq x_i, y_i \leq 10^6)$ 。

接下来的 q 行，每行两个浮点数 $a_i, b_i (0 \leq a_i, b_i \leq 10^6)$ 。

输出

对于每组询问，你需要输出一行一个整数 J_i ，表示 P 中距离 (a_i, b_i) 近似最近的点的下标。

样例输入

```
3 3
0 0
0 1
2 0
0.5 0.5
0.25 0.75
1.5 0.5
```

样例输出

```
1
2
3
```

2. 思路

二叉树：我们每次把一个区域分成四个子区域，直到每个区域足够小为止。具体建树的过程可以使用递归。

插入过程就像给点找“家”：

- 1) 先看当前这个大方格有没有“户主”（代表点），没有的话这个点就当户主
- 2) 如果方格还不够小，就把它切成四个小方格
- 3) 看点属于哪个小方格，就把它送到那个小方格去

4) 重复这个过程，直到方格足够小

查询过程是这个程序最巧妙的地方：

- 1) 我们不是遍历所有点，而是从根节点开始，一层一层检查方格
- 2) 每次检查一个方格时，先看看它的代表点是不是更近
- 3) 然后用剪枝技巧：如果我们已经找到的最近距离，比这个方格内可能的最小距离还要小，那这个方格里面肯定没有更近的点，直接跳过

3. 代码实现

```
#include <bits/stdc++.h>
using namespace std;
struct Node{
    double xMin, yMin, size;
    int rep = -1;
    int child[4]; // -1 means no child
    Node(double _xMin = 0, double _yMin = 0, double _size = 0)
    {
        xMin = _xMin;
        yMin = _yMin;
        size = _size;
        rep = -1;
        for (int i = 0; i < 4; i++)
            child[i] = -1;
    }
};
struct Pts{
    int x, y;
};
vector<Node> tree;
void insert(int nodeIdx, int ptIdx, const vector<Pts> &pts){
    // 1. 把一个点加进去。
    if (tree[nodeIdx].rep == -1)
        tree[nodeIdx].rep = ptIdx;
    // 2. 太小的格子就不动了
    if (tree[nodeIdx].size < 0.1)
        return;
    // 3. 确定新的边界，田字形切分
    double midX = tree[nodeIdx].xMin + tree[nodeIdx].size / 2.0;
    double midY = tree[nodeIdx].yMin + tree[nodeIdx].size / 2.0;
    int quad = 0; // 象限编号：0(左下), 1(右下), 2(左上), 3(右上)
    if (pts[ptIdx].x >= midX)
        quad += 1;
    if (pts[ptIdx].y >= midY)
        quad += 2;
    // 这是AI告诉我的一个超级巧妙的方法。

    // 建造小方格
    if (tree[nodeIdx].child[quad] == -1)
    {
        double nx;
        if (quad % 2 == 1)
            nx = midX;
        else
            nx = tree[nodeIdx].xMin;
        double ny;
        if (quad / 2 == 1)
```

```

        ny = midY;
    else
        ny = tree[nodeIdx].yMin;
    tree.push_back(Node(nx, ny, tree[nodeIdx].size / 2.0));
    tree[nodeIdx].child[quad] = tree.size() - 1; //将原来的树与新生成的树产生联系
}
//递归
insert(tree[nodeIdx].child[quad], ptIdx, pts);
}

double dis(double x1, double y1, double x2, double y2){
    return sqrt((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2));
}

int main()
{
    int n, q;
    cin >> n >> q;
    vector<Pts> pts(n);
    for(int i = 0; i < n; i++){
        cin >> pts[i].x >> pts[i].y;
    }
    tree.push_back(Node(0, 0, 1000001)); // 初始化根节点，覆盖所有点
    for (int i = 0; i < n; i++){
        insert(0, i, pts);
    }
    // 查询
    for(int i = 0; i < q; i++){
        double queryX, queryY;
        cin >> queryX >> queryY;
        int bestIdx = tree[0].rep;
        double minDist = dis(queryX, queryY, pts[bestIdx].x, pts[bestIdx].y);
        vector<int> todo;
        todo.push_back(0);
        int head = 0;
        while(head < todo.size()){
            int curr = todo[head++];
            for(int j=0; j<4; j++){
                int c = tree[curr].child[j];
                if(c == -1) continue;
                // 看看这个子方格的代表点
                int p_idx = tree[c].rep;
                double d = dis(queryX, queryY, pts[p_idx].x, pts[p_idx].y);
                // 更新我们找到的最短距离
                if(d < minDist){
                    minDist = d;
                    bestIdx = p_idx;
                }
                // 核心：剪枝（如果方格太远，就不管它了）
                // 方格直径约等于 size * 1.414
                if(minDist > 1.15 * (d - tree[c].size * 1.414))
                    todo.push_back(c);
            }
        }
        cout << bestIdx << endl;
    }
    return 0;
}

```